- Content Virality Platform
  - Key Features
    - 🔐 Multi-User Authentication System
    - 📊 Analytics & Predictions
    - ⚙️ Administrative Features
  - Tech Stack
  - Project Structure
  - Getting Started
    - 1. Install Dependencies
    - 2. Initialize Database
    - 3. Run the Application
    - 4. Access the Platform
  - Configuration
  - Development Notes

Content Virality Platform

A comprehensive web-based analytics system built with Python and Streamlit that provides content virality prediction, topic analysis, and administrative management capabilities.

Key Features

🔐 Multi-User Authentication System

- **User Registration & Login**: Secure account creation and authentication
- **Role-Based Access Control**: System Administrators, Content Strategists, Users, and Viewers
- **Profile Management**: Users can update their personal information and change passwords
- **Session Management**: Secure session handling with automatic timeout

📊 Analytics & Predictions

- **Virality Scoring**: AI-powered headline analysis with trend relevance scoring
- **Topic Explorer**: Search and analyze Reddit content with interactive visualizations
- **Dashboard**: Real-time trending topics with filtering and sorting capabilities
- **Data Export**: CSV and PDF export functionality for reports and analysis

⚙️ Administrative Features

- **User Management**: Activate/deactivate accounts and assign roles
- **Data Source Management**: Configure and monitor external data sources
- **ML Model Management**: Model versioning, performance monitoring, and retraining
- **Activity Logging**: Comprehensive audit trails for all system actions
- **System Monitoring**: Real-time system health and performance metrics

Tech Stack

- Python 3.10+
- Streamlit for UI
- scikit-learn (initial modeling baseline)
- pandas, numpy for data
- plotly/altair/matplotlib for visualization
- Optional: PRAW/Pushshift/Reddit API, requests/bs4 for scraping

Project Structure

```
.
├─ README.md
├─ requirements.txt
├─ .streamlit/
│  └─ config.toml
└─ src/
   ├─ app.py                      # Streamlit entry point (today's focus: UI)
   ├─ ui/
   │  ├─ __init__.py
   │  └─ components.py            # Reusable UI components
   ├─ data/
   │  ├─ __init__.py
   │  ├─ scraping/
   │  │  └─ reddit_scraper.py     # Reddit scraping service (stub)
```

```
│   └── processing/
│       └── preprocess.py          # Cleaning & feature engineering (stub)
├── models/
│   ├── __init__.py
│   ├── virality_predictor.py      # Model APIs (stub)
│   └── topic_model.py             # Topic modeling APIs (stub)
├── services/
│   ├── __init__.py
│   ├── config.py                  # Centralized settings
│   └── cache.py                   # Caching layer (stub)
└── utils/
    ├── __init__.py
    └── logger.py                  # Structured logging
```

Getting Started

1. Install Dependencies

Create a virtual environment and install dependencies:

```
python -m venv .venv
.venv\Scripts\activate  # On Windows
# or
source .venv/bin/activate  # On macOS/Linux

pip install -r requirements.txt
pip install -r requirements-nlp.txt
```

2. Initialize Database

Set up the database with default roles and admin user:

```
python init_db.py
```

This creates:

- Database tables for users, roles, data sources, and models
- Default roles (System Administrator, Content Strategist, User, Viewer)
- Admin user with credentials: `admin` / `admin123`
- strategist credentials: `strategist / strategist123`

- viewer credentials: `viewer / viewer123`

3. Run the Application

```
cd src
streamlit run app.py
```

4. Access the Platform

Open your browser and go to `http://localhost:8501` (or the URL shown in terminal).

Default Login:

- Username: `admin`
- Password: `admin123`

⚠️ **Important**: Change the default admin password after first login!

Configuration

- UI/theme config lives in `.streamlit/config.toml`.
- App settings and constants in `src/services/config.py`.

Development Notes

- Today's milestone focuses on a polished Streamlit UI, wired to mock data so the app is immediately runnable.
- Modeling, scraping, and pipelines are stubbed for fast iteration in the coming days.